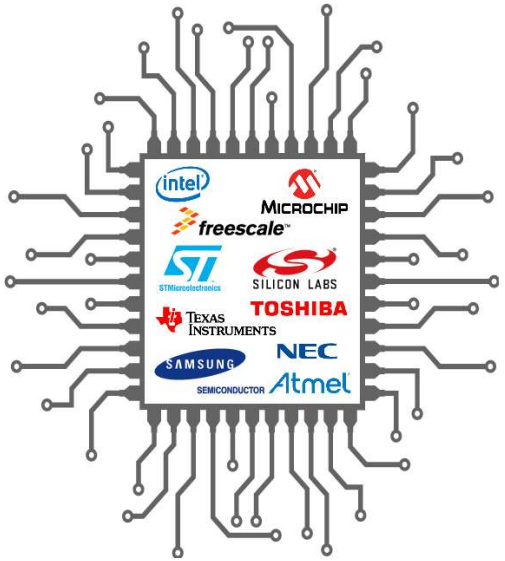
Serial Communication



Thareendhra Keerthi Wijayasiriwardhane

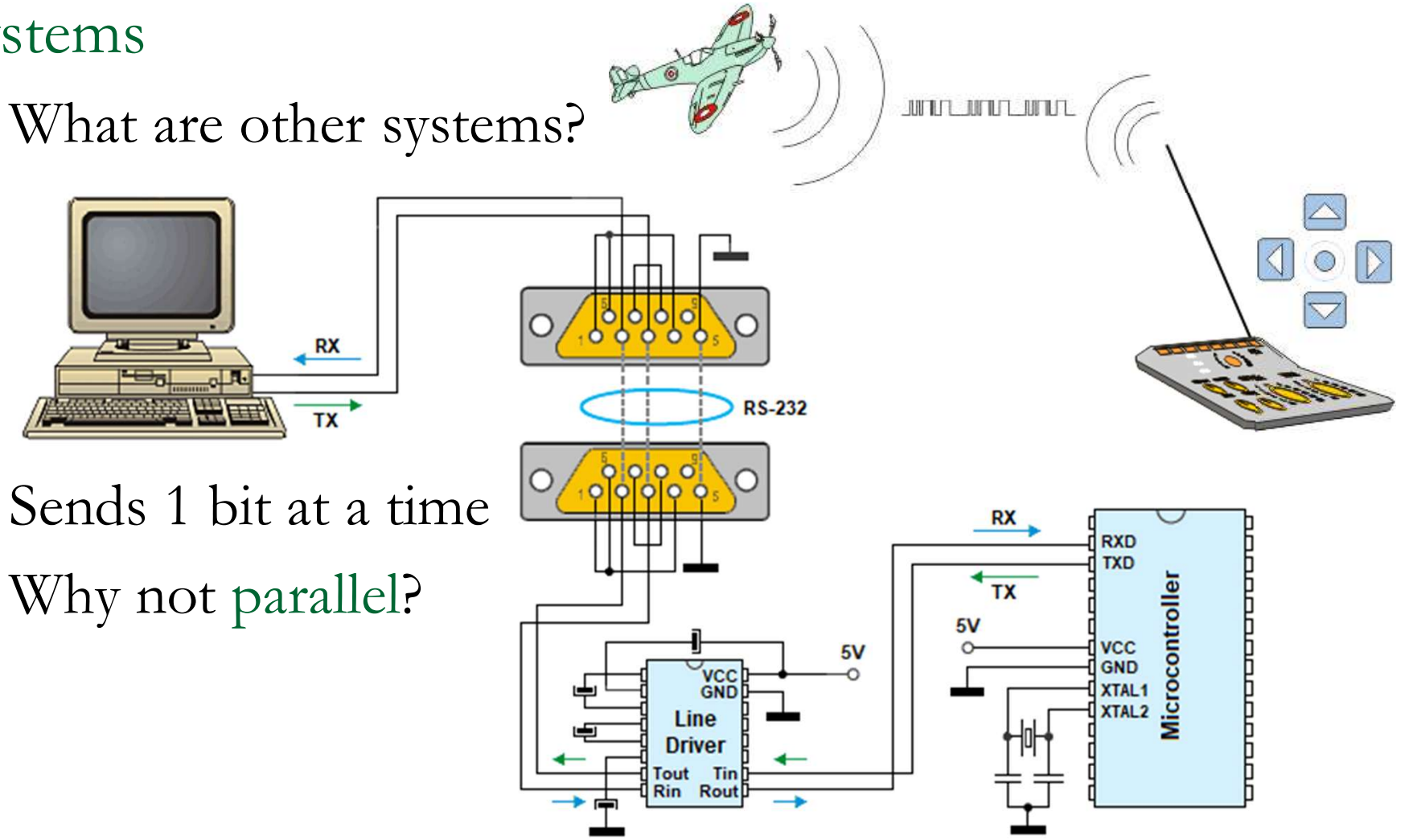
Department of Industrial Management
Faculty of Science
University of Kelaniya
thareen@kln.ac.lk

Learning Outcomes

- On completion of this session, you should be able to
  - Discuss serial vs. parallel communications in the context of embedded systems
  - Contrast asynchronous vs. synchronous communications
  - Explain asynchronous data transfer protocol
  - Explain need for Line Drivers/USART to USB interfaces
  - Explain how Tx and Rx transmit and receive data
  - Explain the functionality of USART registers
  - Construct a microcontroller-based system and program it to communicate with another system

Serial Communication

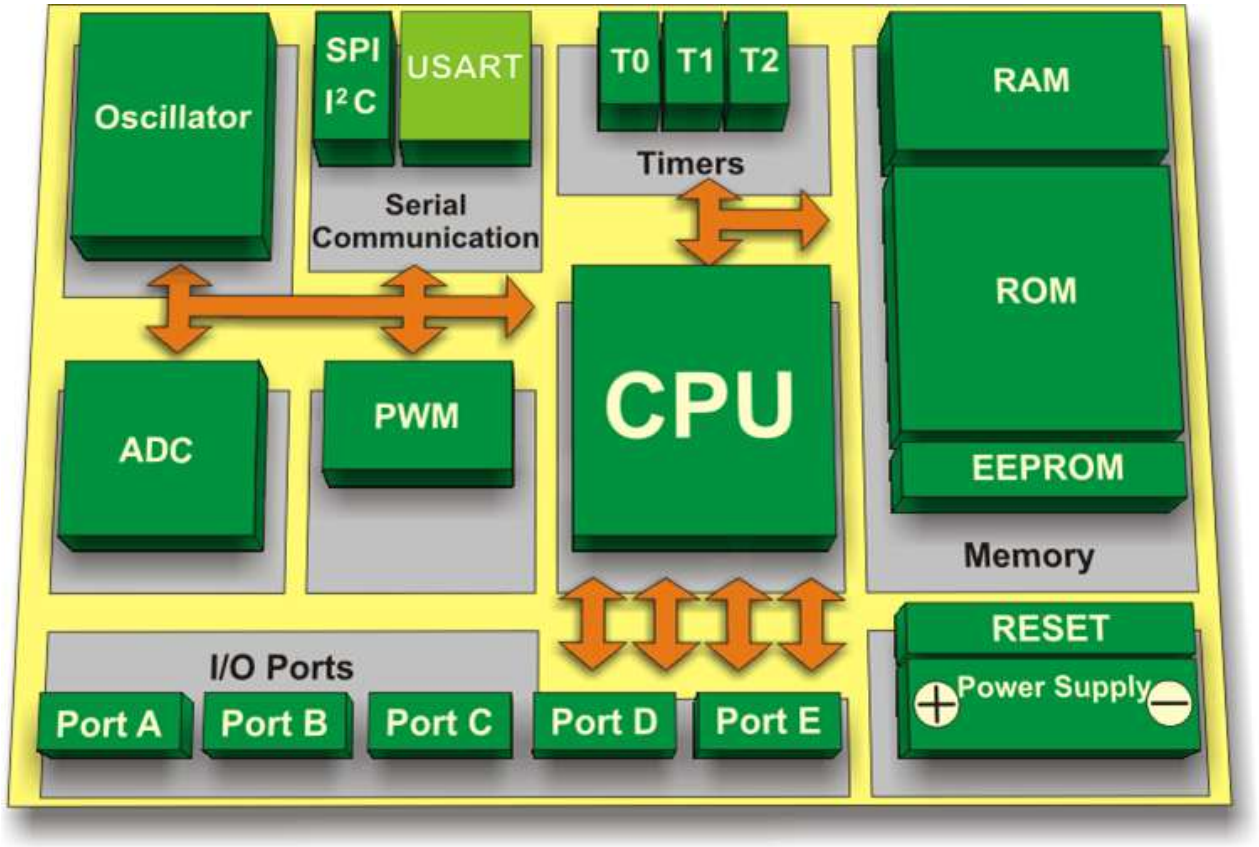
- Used by embedded systems to communicate with other systems
  - What are other systems?



  - Sends 1 bit at a time
  - Why not parallel?

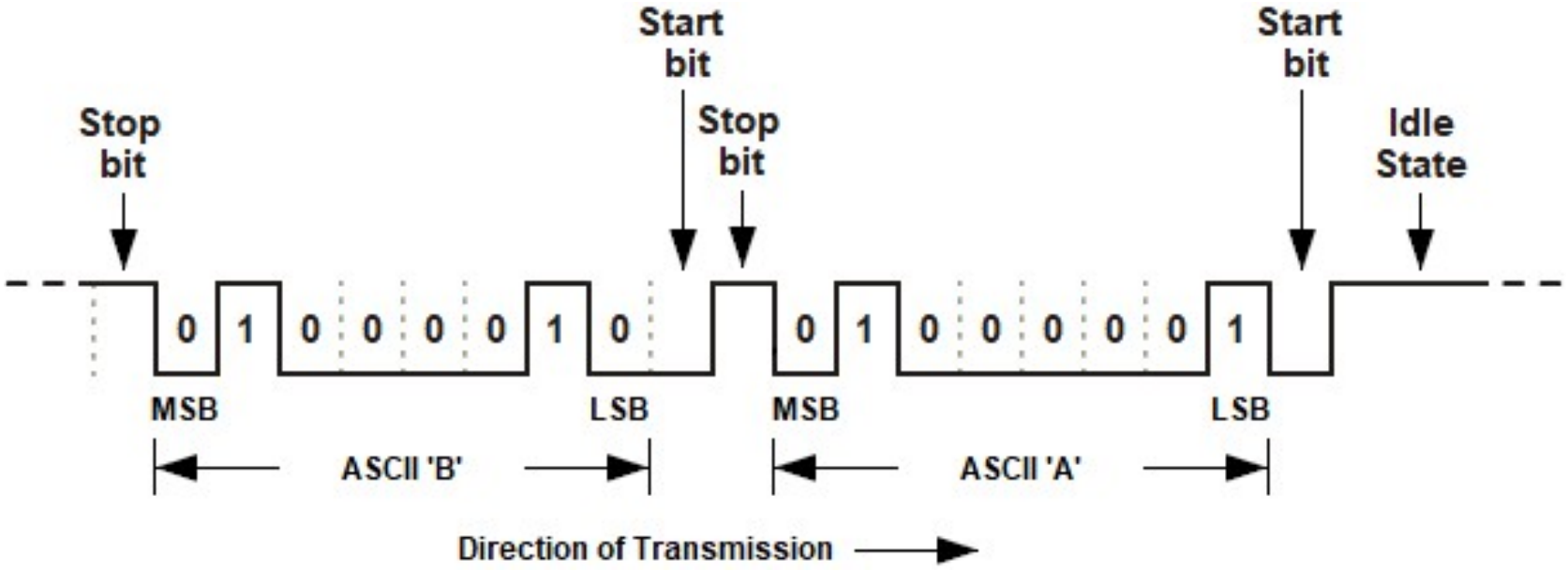
USART

- A peripheral device that implements both asynchronous and synchronous serial communications
  - Asynchronous
  - Synchronous
- Why separate peripheral device for data communication?



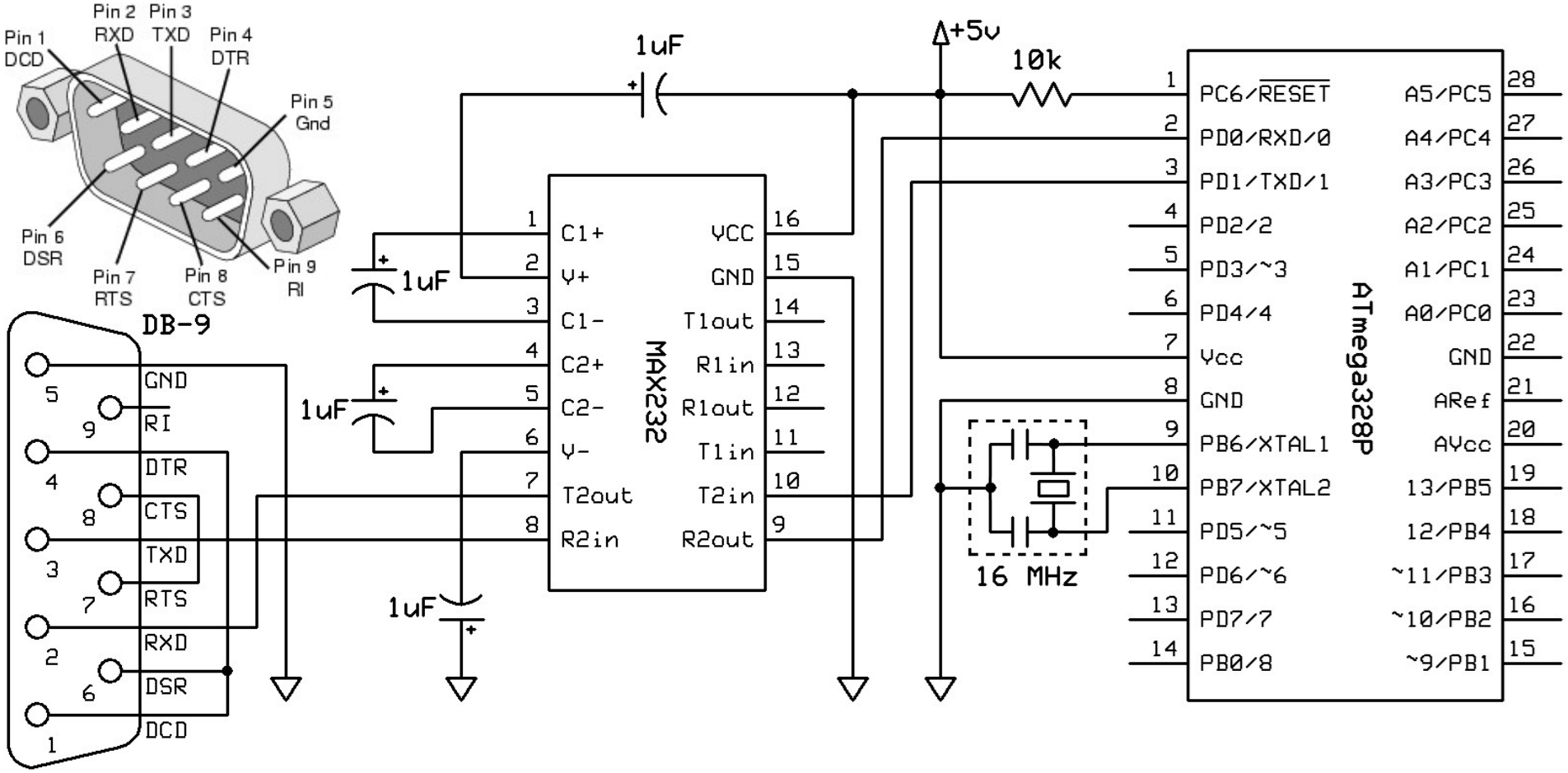
Asynchronous Data Transfer

- Each character is framed between a start bit and a stop bit and then transmitted
- Why baud rate is important?
- How can receiver recognize transmitted characters?



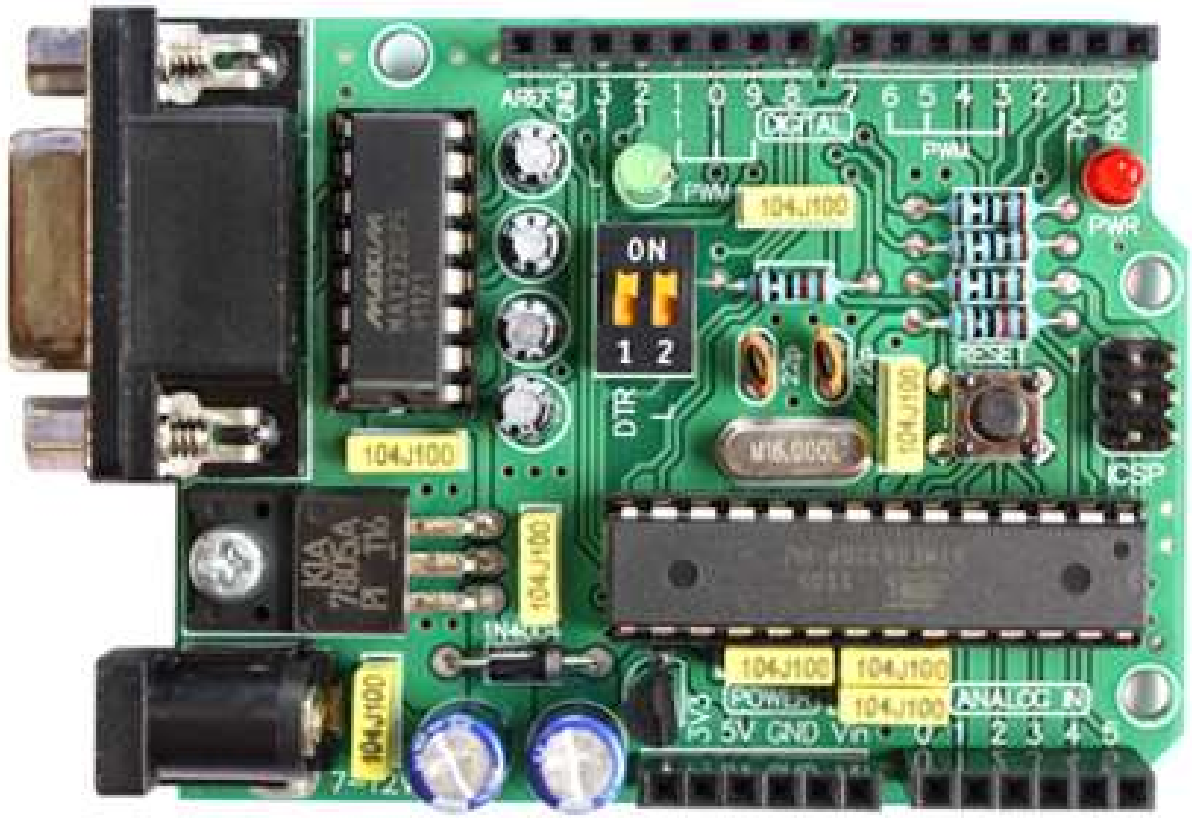
Connection over Serial Port

- Using a Line Driver
  - Why not connect directly?



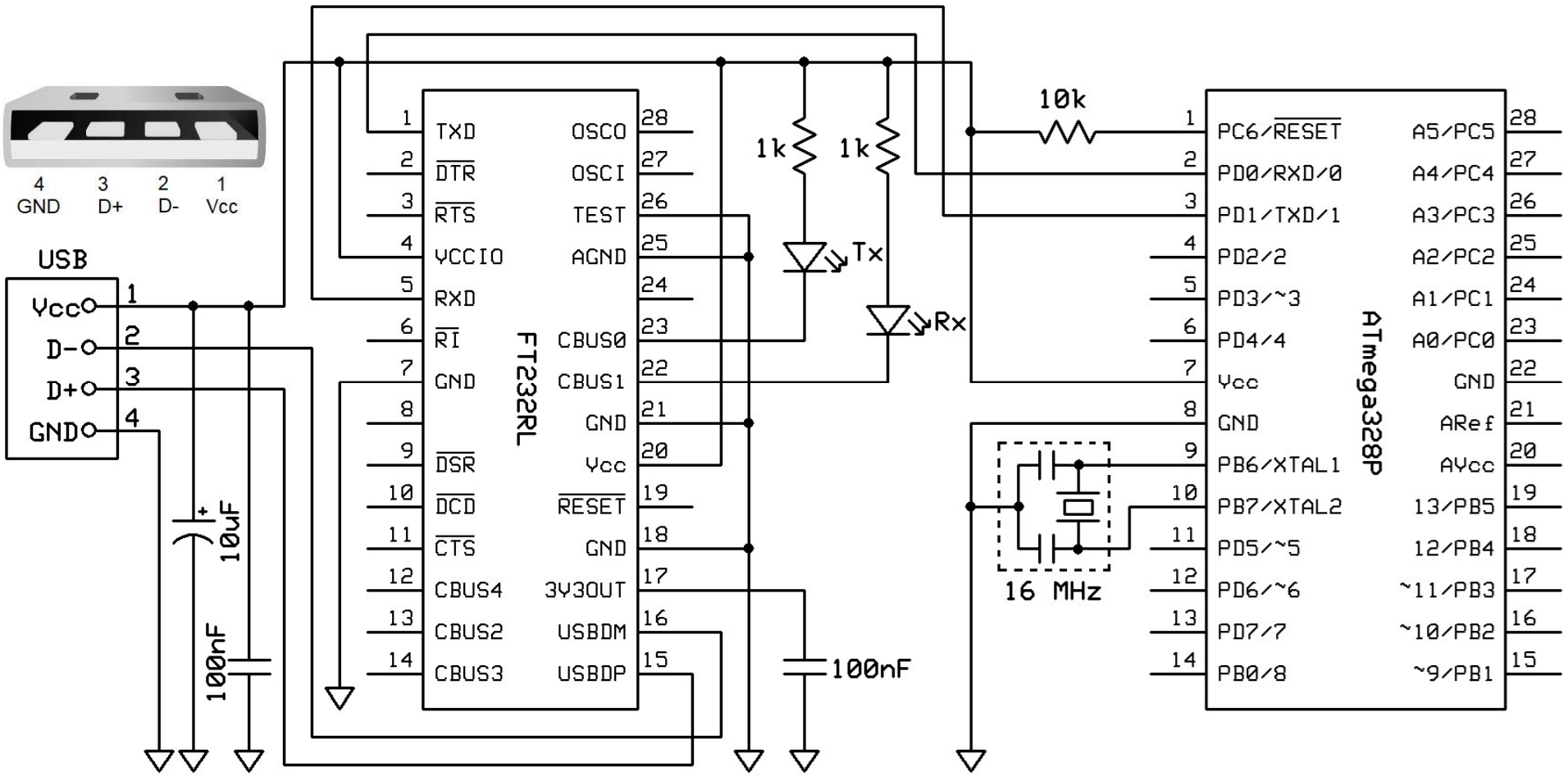
Arduino Severino

- ATmega328P with MAX232



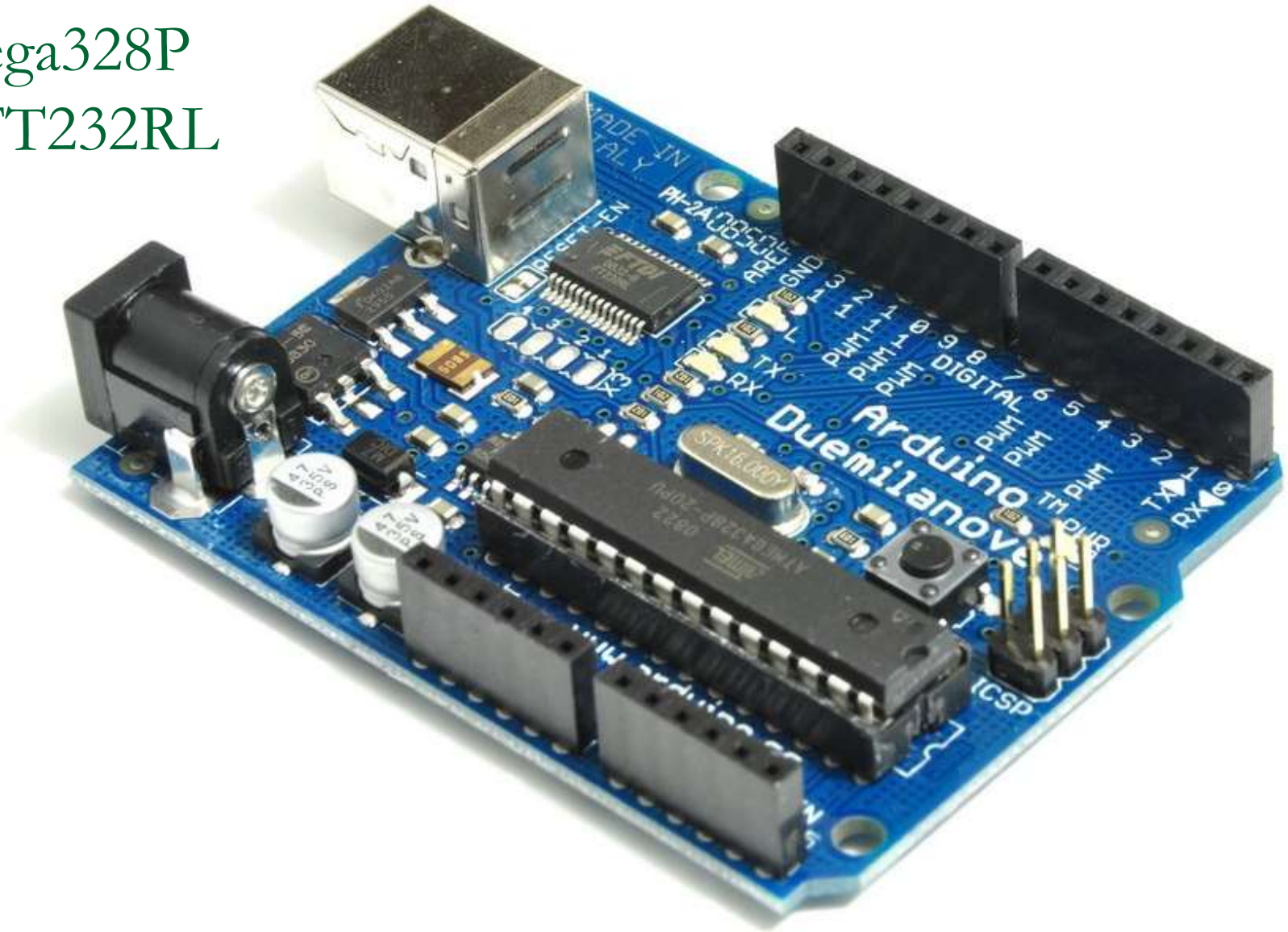
Connection over USB Port

- Using a UART to USB interface controller



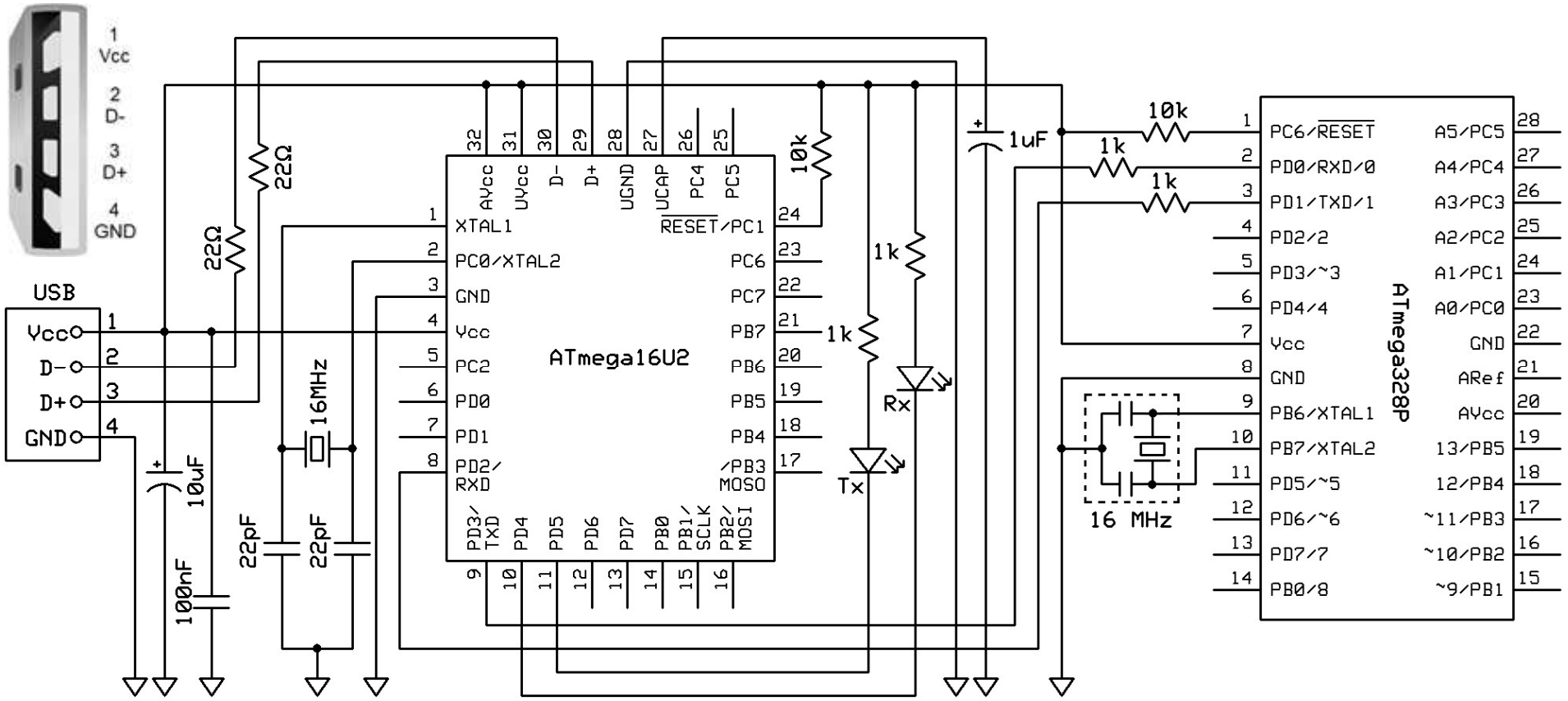
Arduino Duemilanove

- ATmega328P
  with FT232RL



Connection over USB Port

- Using another Microcontroller as UART to USB interface controller

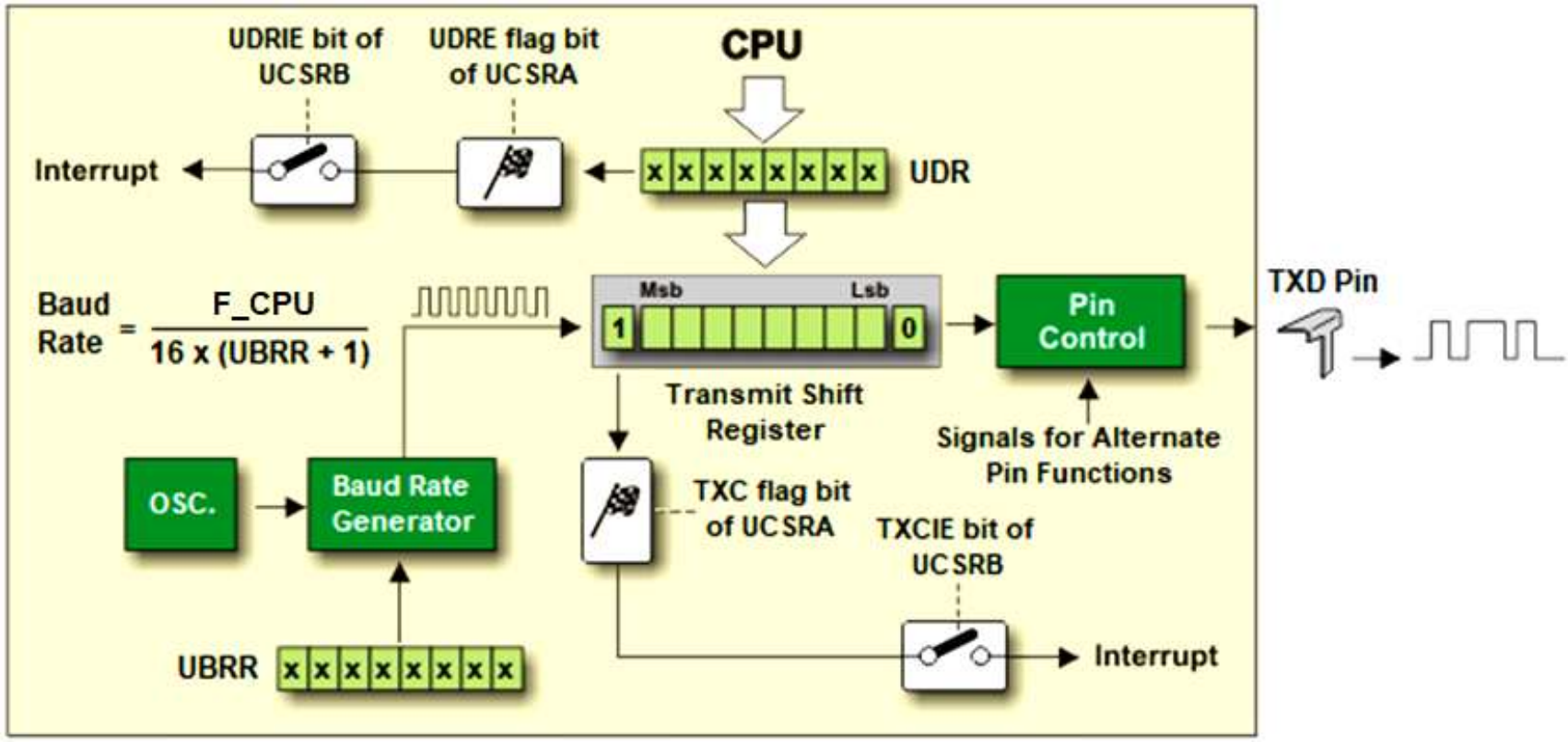


Arduino Uno

- ATmega328P
  with ATmega16U2

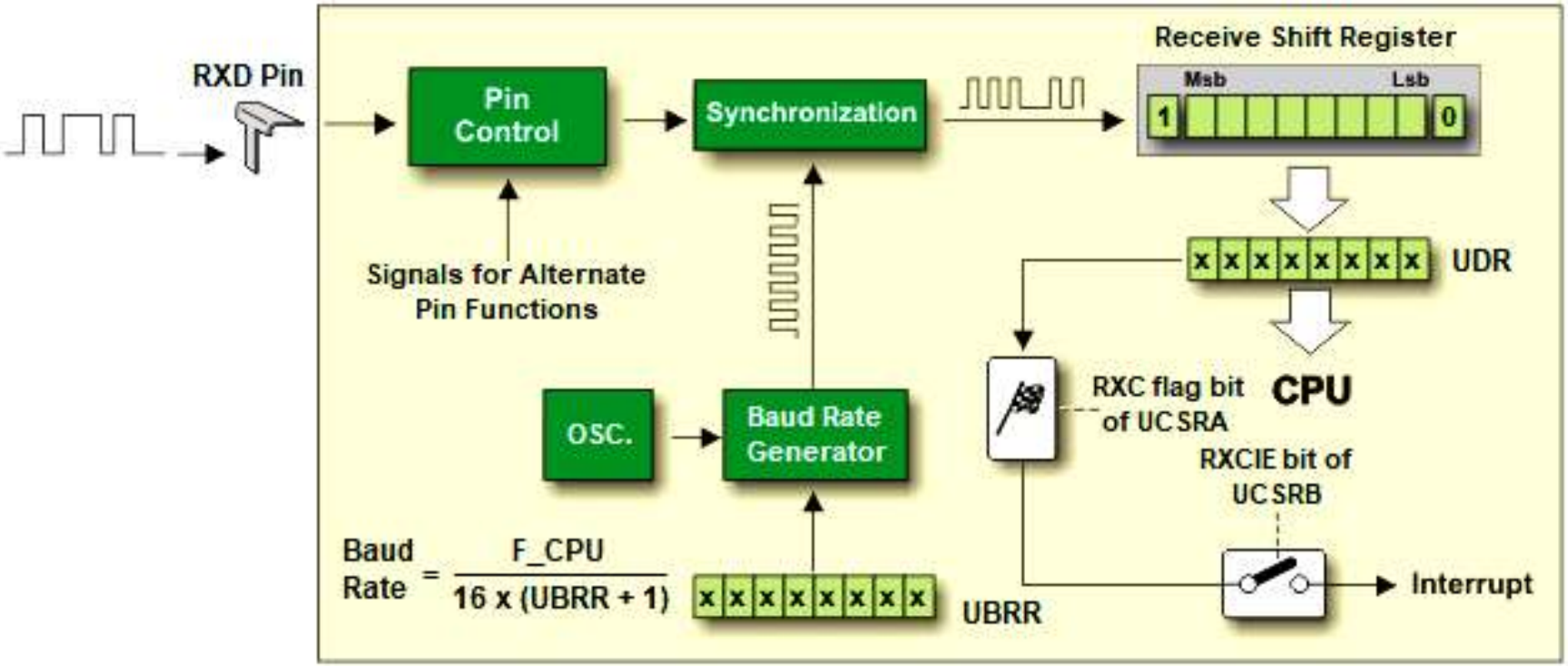


Asynchronous Transmitter



- Which flag (UDRE or TXC) gives maximum throughput?
- What's the purpose of TXC flag?

Asynchronous Receiver



- What's the role of Pin Control?
- What's the need of Shift Registers?

USART Registers

- UBRRn (USART Baud Rate Register n)
  - Used to set baud rate
  - 16-bit register but only lower 12 bits are used

$$Baud\ rate = \frac{F_CPU}{16 \times (UBRR + 1)}$$

$$UBRR = \left(\frac{F_CPU}{16 \times Baud\ rate}\right) - 1$$

USART Registers

- UDRn (USART Data Register n)
  - Refers to Transmit Data Buffer that holds the character to be transferred to Transmit Shift Register
  - Also refers to Receive Data Buffer that holds the character transferred from Receive Shift Register

USART Registers

- UCSRnA (USART Control and Status Register n A)



UCSRnA	RXCn	TXCn	UDREn	n/a	n/a	n/a	n/a	n/a	Bit name
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	

  - RXCn (Receive Complete) flag bit is set when UDRn contains unread data
  - UDREn (USART Data Register Empty) flag bit is set when UDRn becomes empty
  - TXCn (Transmit Complete) flag bit is set when entire frame in Transmit Shift Register has been shifted out and UDRn is empty

USART Registers

- UCSRnB (USART Control and Status Register n B)



UCSRnB	RXCIEn	TXCIEn	UDRIEn	RXENn	TXENn	UCSZn2	n/a	n/a	Bit name
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	

  - RXCIEn (Receive Complete Interrupt Enable) bit and TXCIEn (Transmit Complete Interrupt Enable) bit enable interrupts on RXCn and TXCn flags, respectively
  - UDRIEn (USART Data Register empty Interrupt Enable) bit enables interrupt on UDREn flag
  - RXENn (Receiver ENable) and TXENn (Transmitter ENable) bits enable RX and TX, respectively
    - What is the need of enabling and disabling RX and TX?

USART Registers

- UCSRnC (USART Control and Status Register n C)



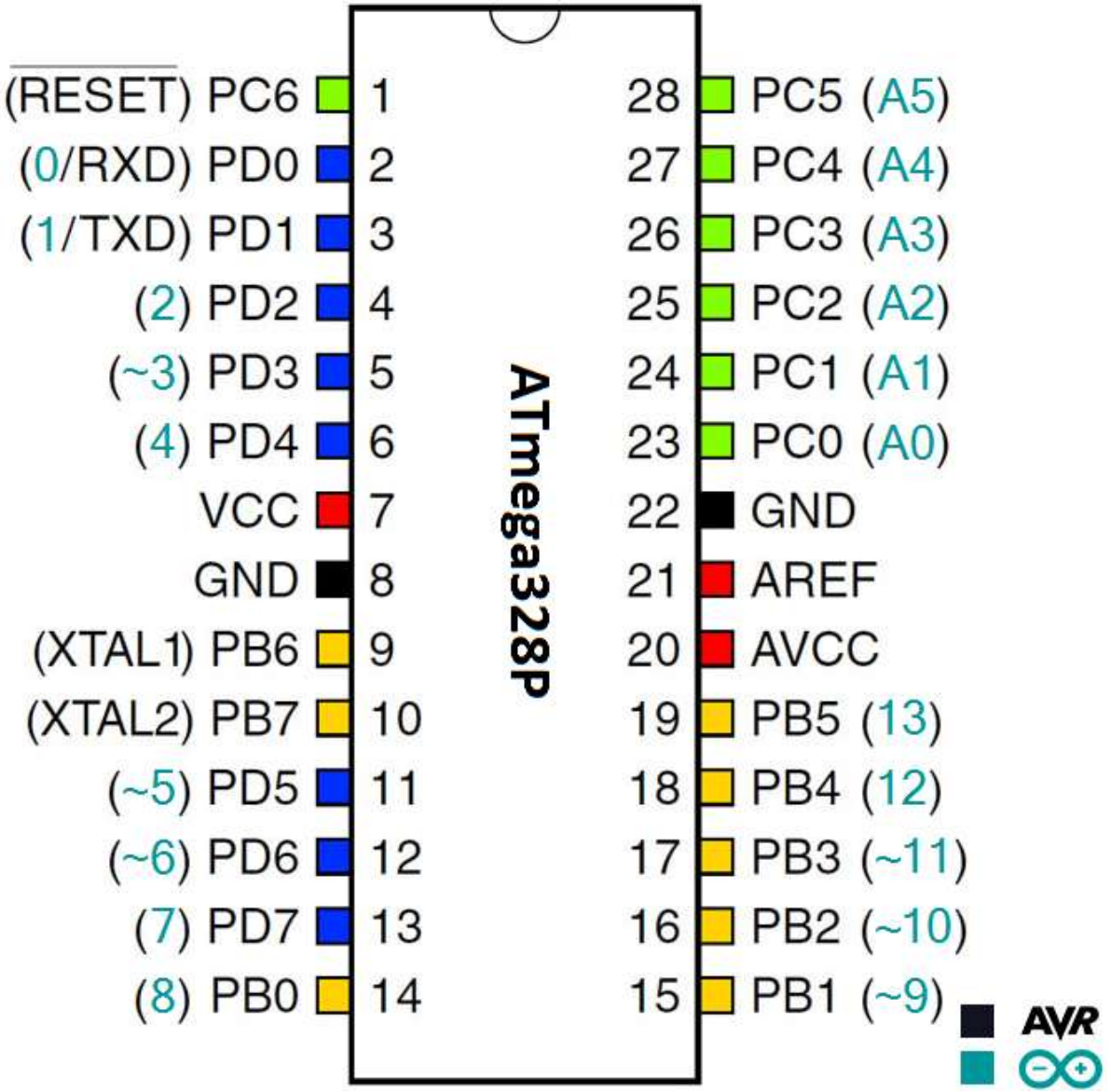
  - UCSZn0 (USART Character SiZe), UCSZn1 bits and UCSZn2 bit of UCSRnB specify character size for communication

UCSZn2	UCSZn1	UCSZn0	Character Size
0	0	0	5-bit
0	0	1	6-bit
0	1	0	7-bit
0	1	1	8-bit
1	1	1	9-bit

System 16

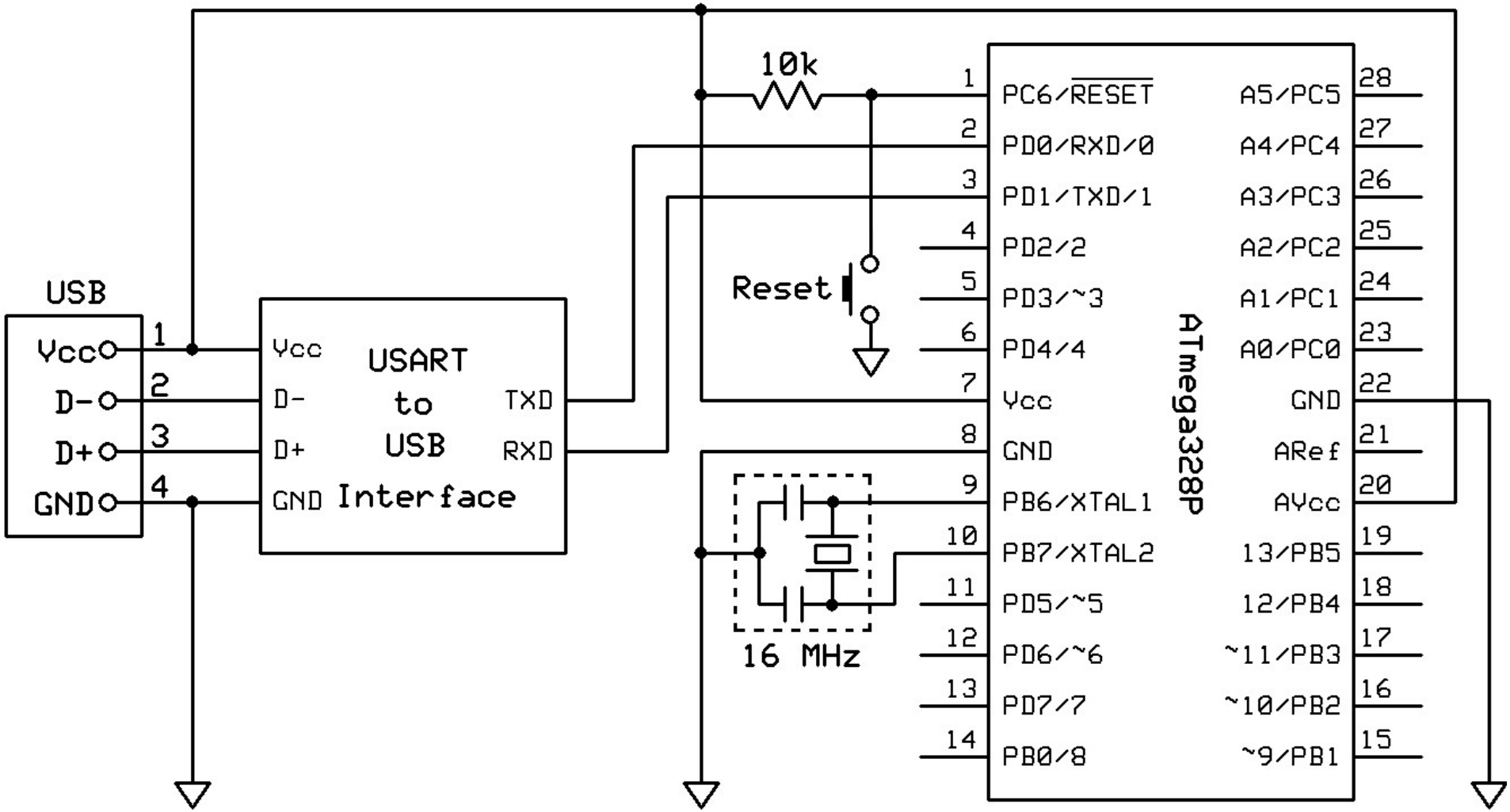
charEncloser - Receives characters from PC, encloses them with square brackets and transmits enclosed characters back to PC

ATmega328P USART Pinout



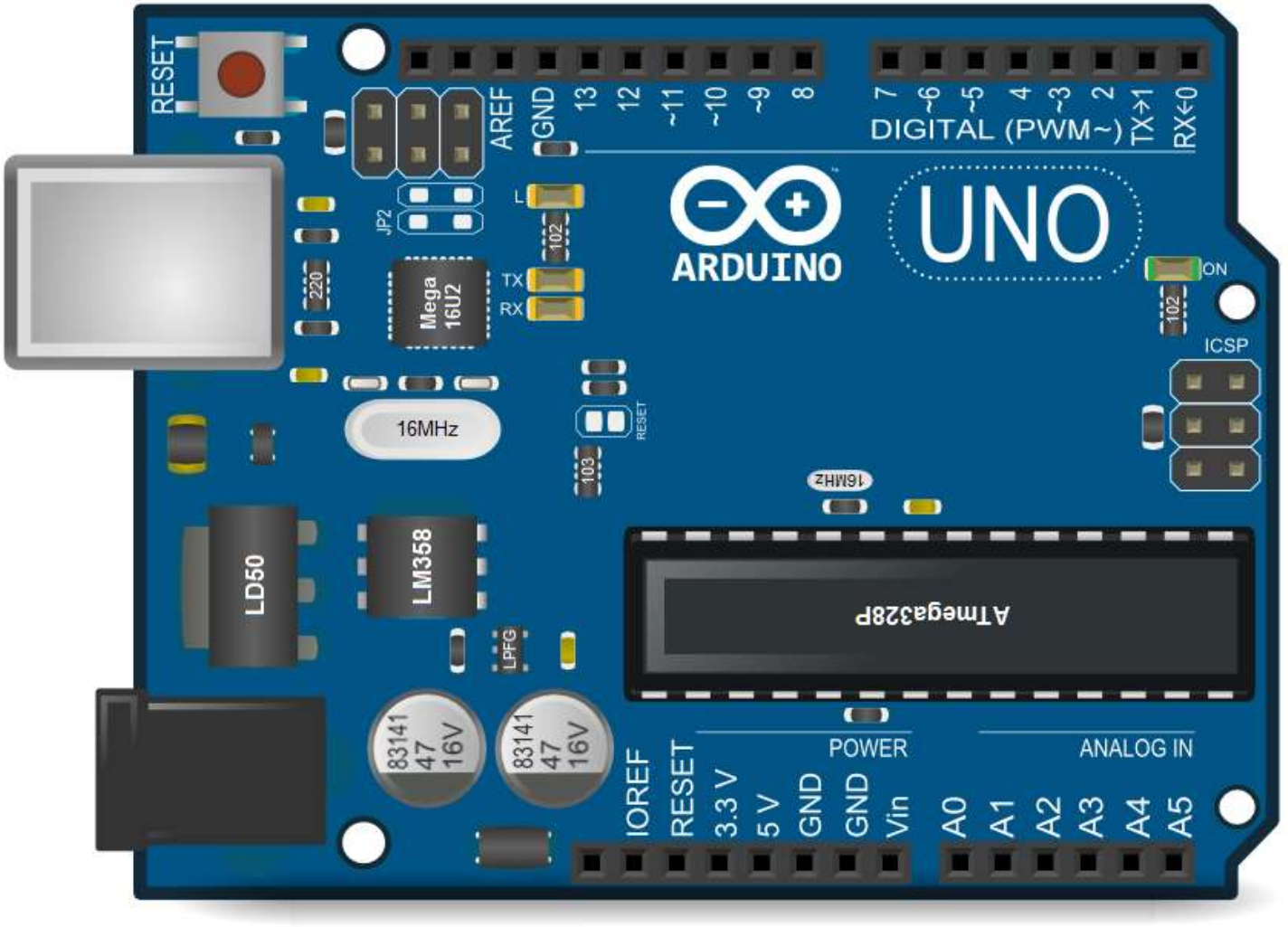
Hardware

- Schematic Diagram



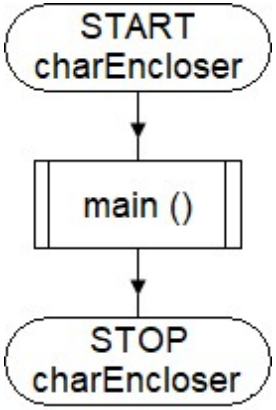
Hardware

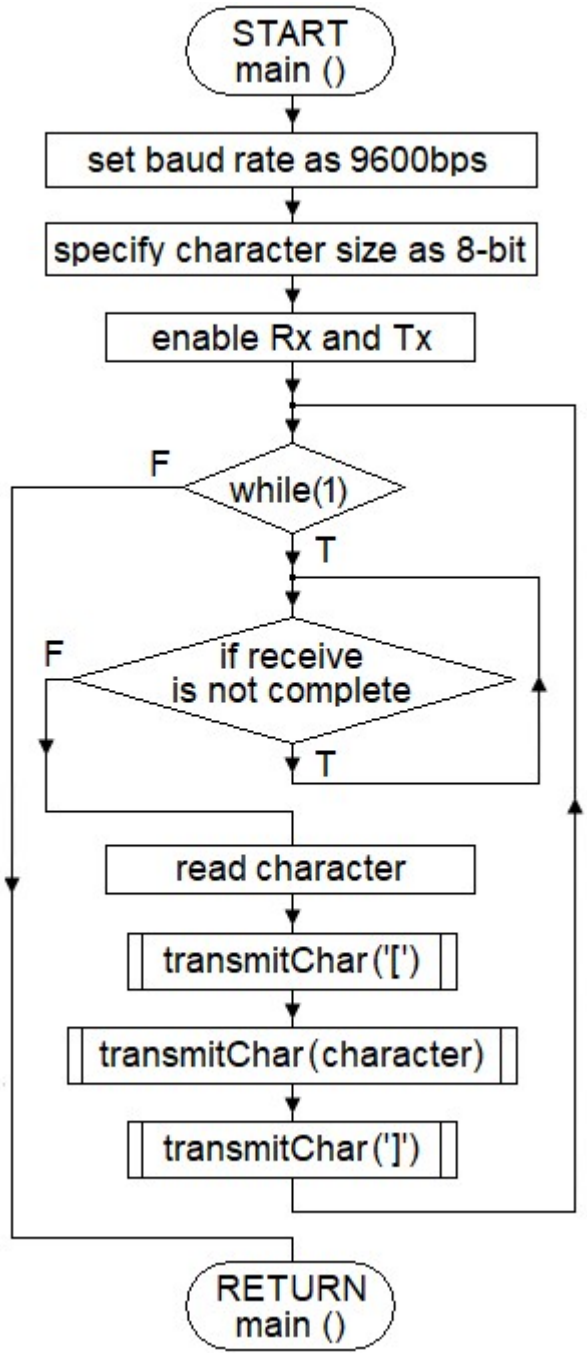
- Wiring Diagram

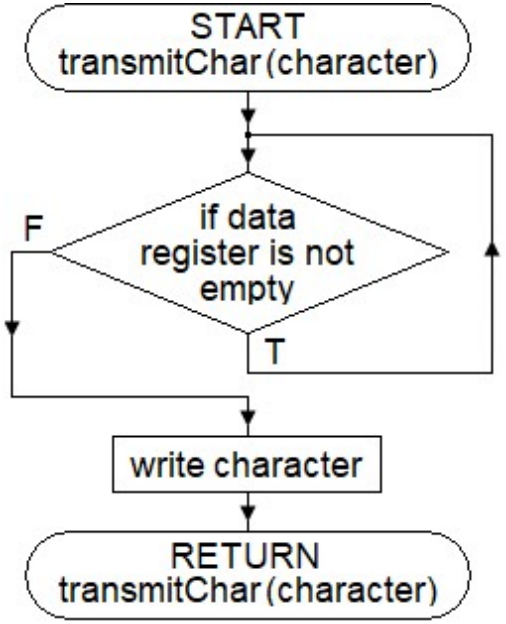


Firmware

- Algorithm







Setting Baud Rate

- Select a standard baud rate from 2400, 4800, 9600, 14400, 19200, 28800, 38400, 57600, 76800, 115200, 230400, 460800, to 921600 bps
  - Why not other values?
- Set the baud rate

$$UBRR = \left(\frac{F_CPU}{16 \times Baud\ rate}\right) - 1$$

  - Determine and set the value of UBRR

```
UBRR0 = 16000000 / (16 * 9600UL) - 1;
```

  - Why do you need to add UL suffix?

Baud Rate Error

- Determine actual baud rate

$$Baud\ Rate = \frac{F_CPU}{16 \times (UBRR + 1)}$$

- Calculate baud rate error

$$Baud\ Rate\ Error = \left(\frac{actual\ BR - desired\ BR}{desired\ BR}\right) \times 100\%$$

- What would happen if a baud rate with higher error % is used?
- How to minimize baud rate error?

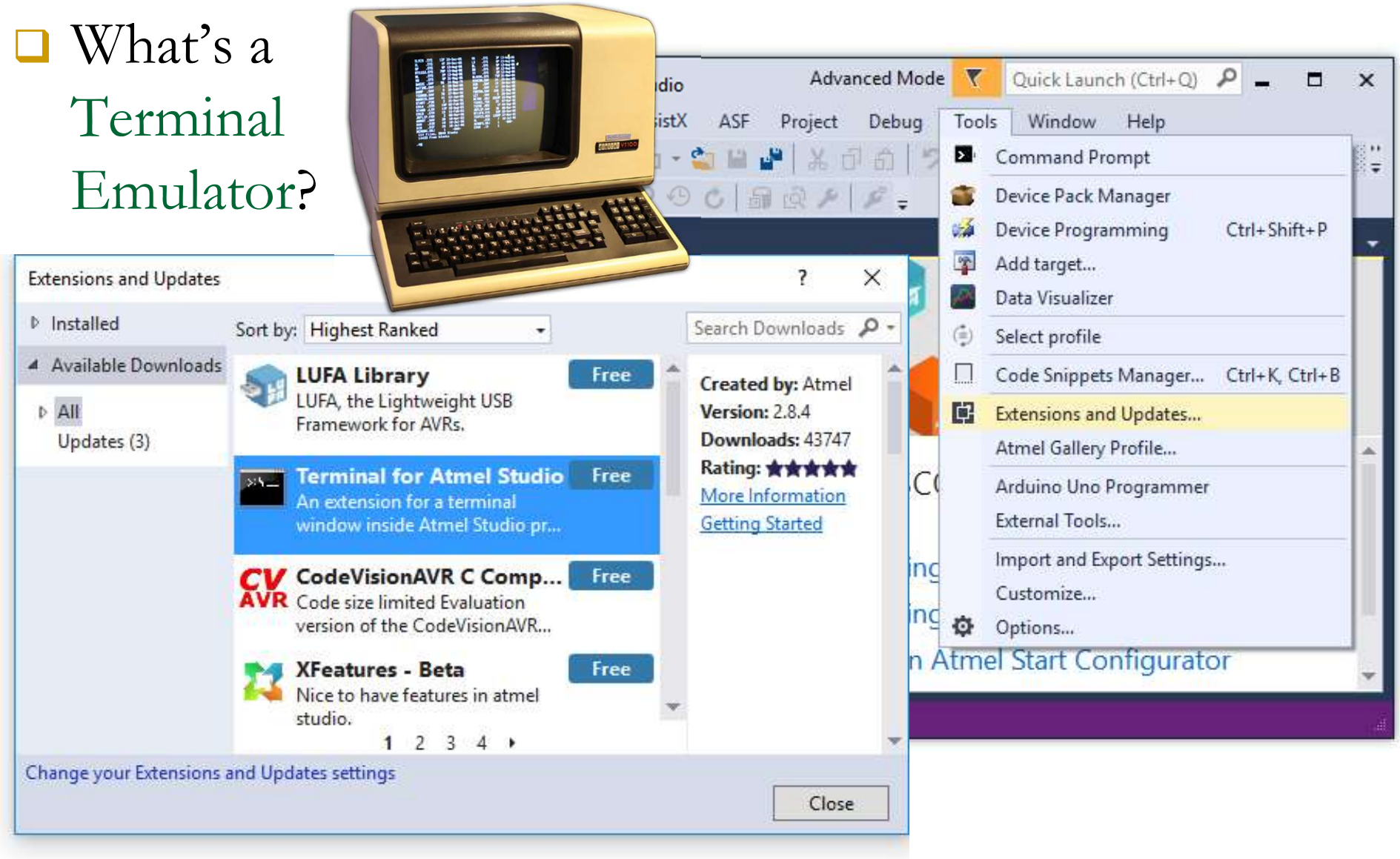


C Program

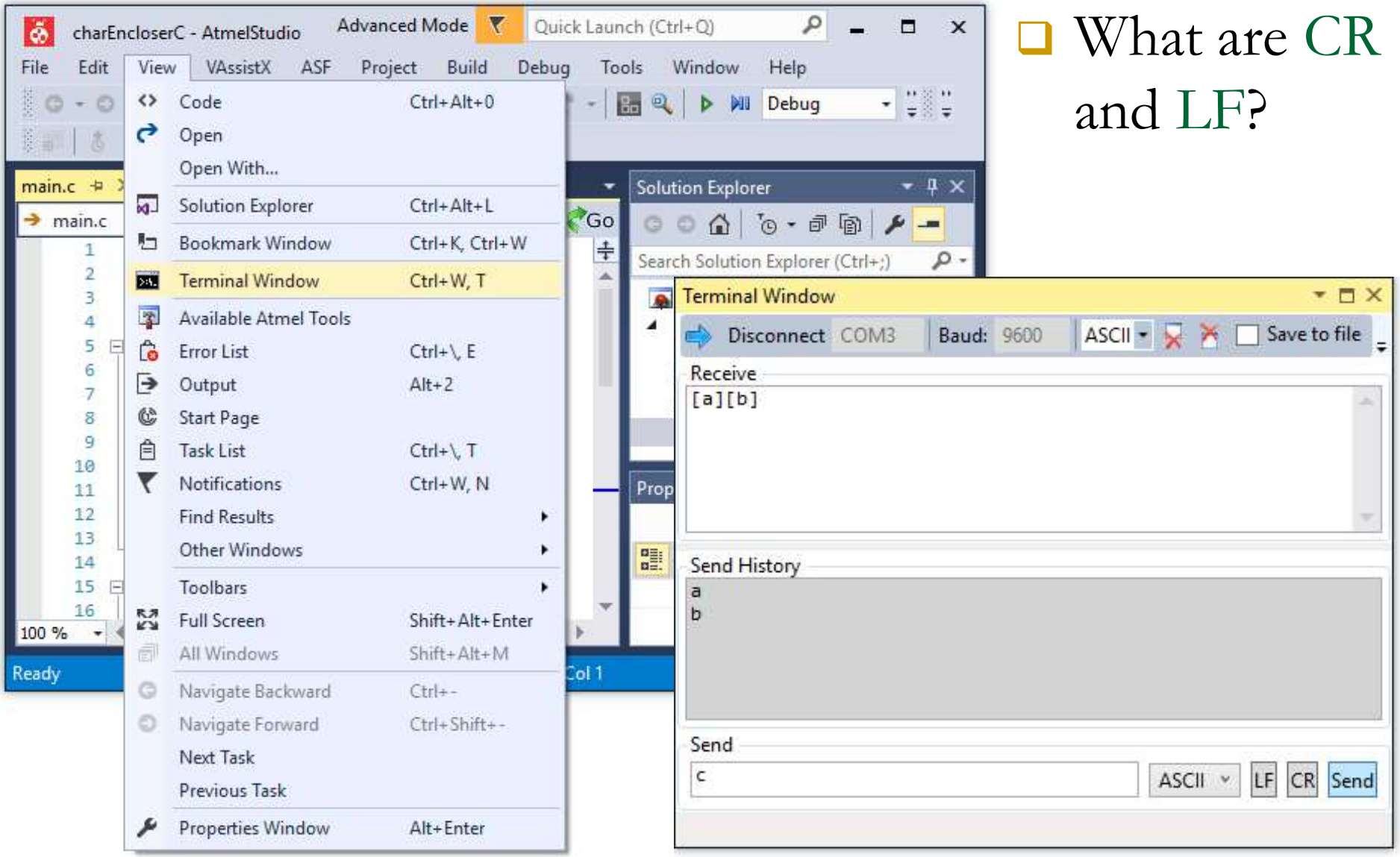
```
1   // charEncloserC.c
2
3   #include <avr/io.h>                          // I/O definitions
4
5   void transmitChar(char ch)                   // transmitChar() subroutine
6   {
7       while(!(UCSR0A & 0b00100000));           // poll for data register empty flag
8       UDR0 = ch;                               // write character to data register
9   }
10
11  int main(void)
12  {
13      UBRR0 = 16000000 / (16 * 9600UL) - 1;    // set baud rate as 9600bps
14      UCSR0B = UCSR0B & 0b11111011;            //
15      UCSR0C = UCSR0C | 0b00000110;            // specify char size as 8-bit
16      UCSR0B = UCSR0B | 0b00011000;            // enable RX & TX
17      while (1)
18      {
19          while(!(UCSR0A & 0b10000000));       // poll for receive complete flag
20          char ch = UDR0;                      // read character from data register
21          transmitChar('[');                   //
22          transmitChar(ch);                    // call transmitChar()
23          transmitChar(']');                   //
24      }
25      return 0;
26  }
```

Extending IDE with a Terminal

- What's a Terminal Emulator?



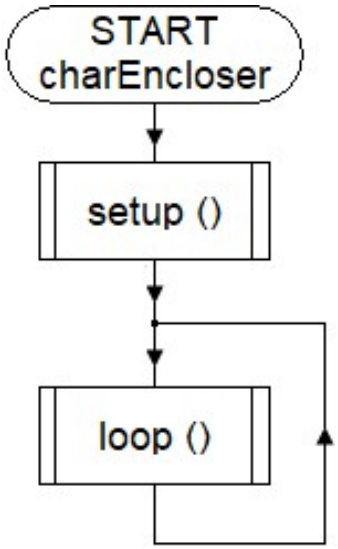
Communicating over Terminal

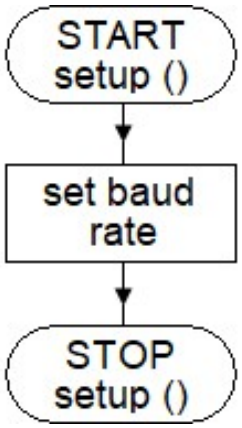


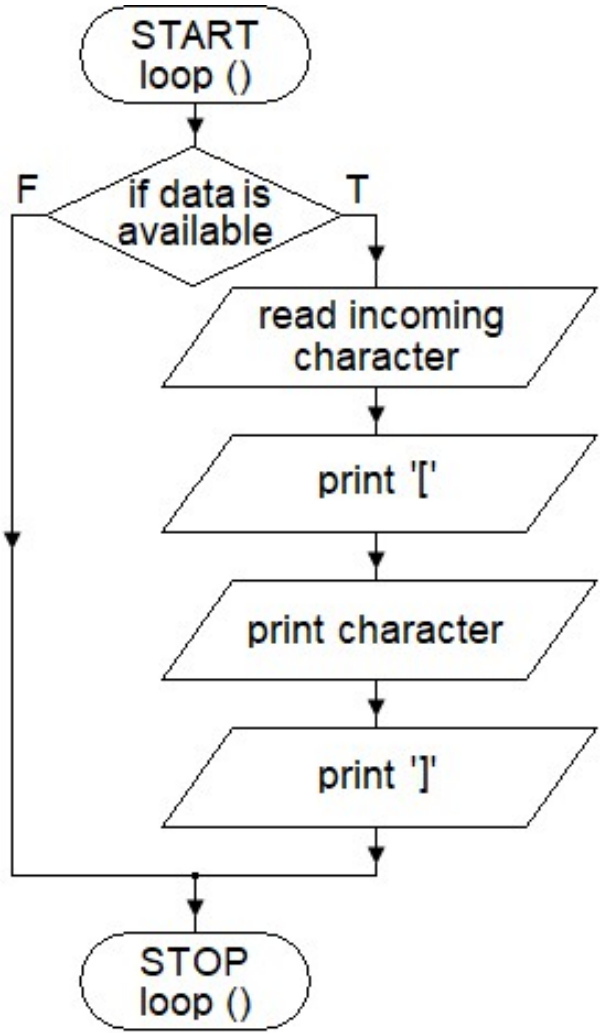
- What are CR and LF?

Firmware

- Algorithm for Arduino







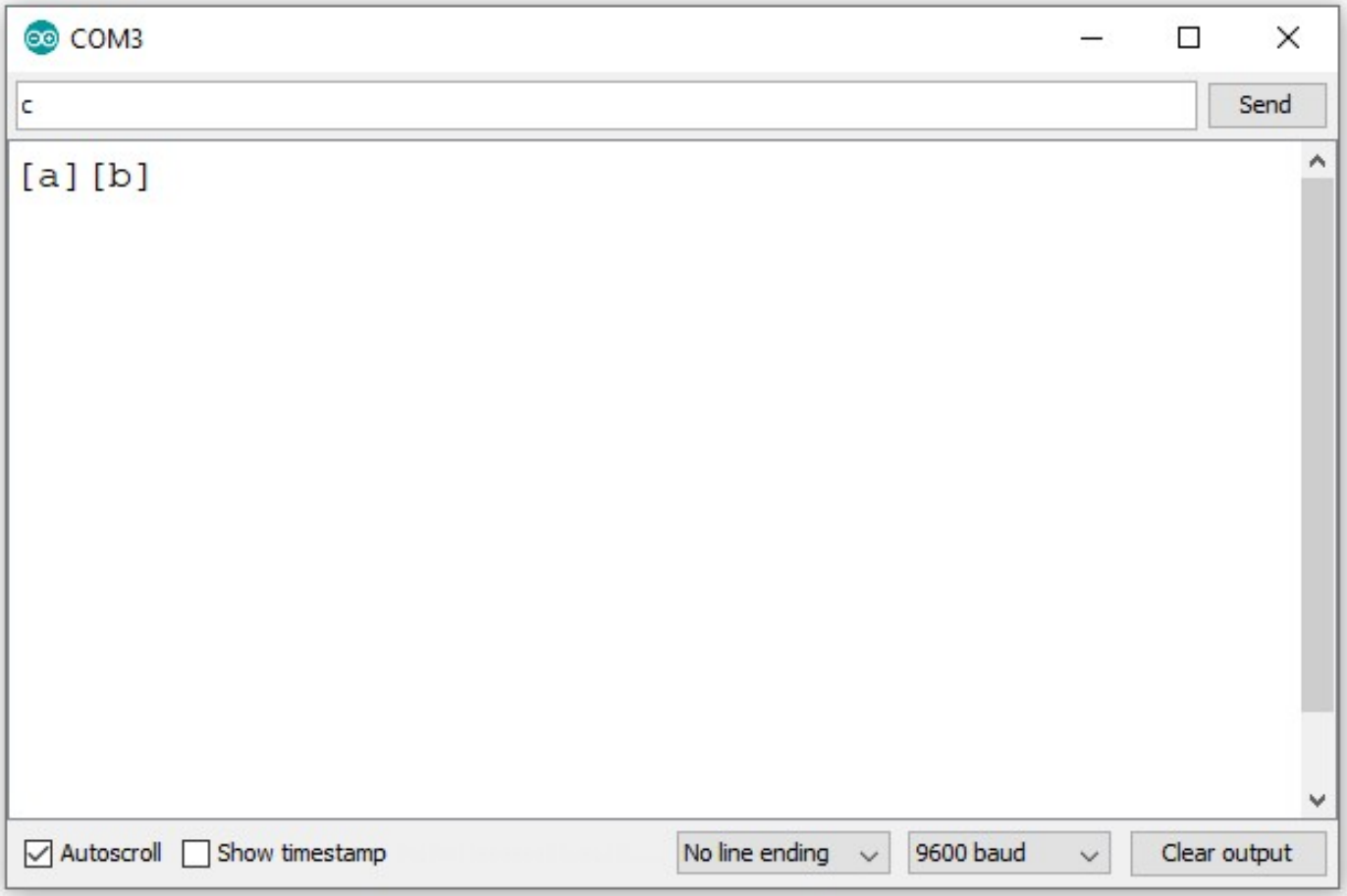
Firmware

- Source Code (using Arduino Wrapper Functions)

```
1 // charEncloser.ino
2
3 void setup()
4 {
5   Serial.begin(9600);          // set baud rate
6 }
7
8 void loop()
9 {
10   if (Serial.available())     // if new data available
11   {
12     char ch = Serial.read();  // read incoming character
13     Serial.print('[');        //
14     Serial.print(ch);         // print enclosed character
15     Serial.print(']');        //
16   }
17 }
```

Serial Monitor

- Terminal emulator of Arduino IDE

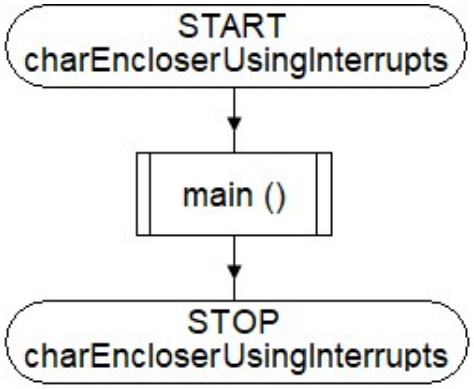


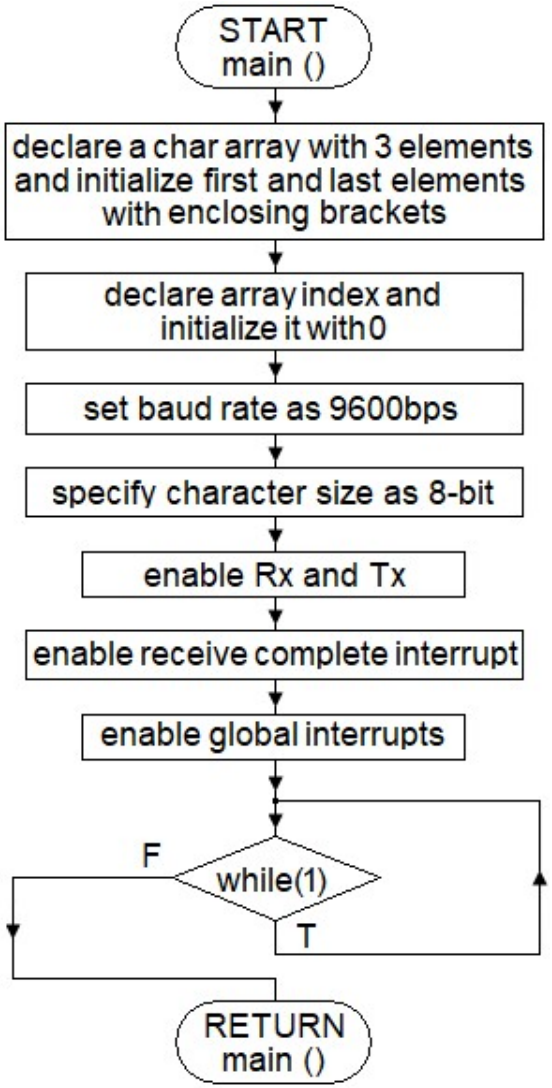
System 17

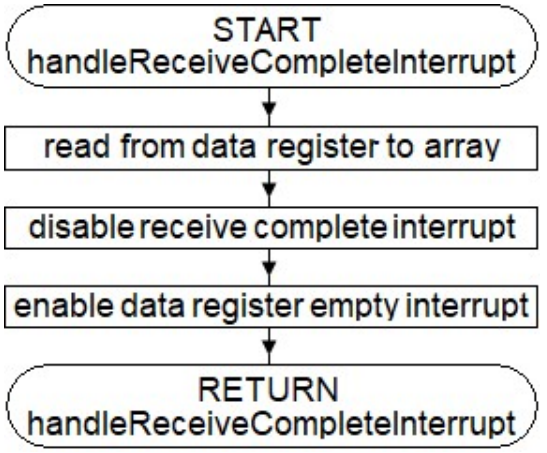
charEncloserUsingInterrupts - Receives characters from PC, encloses them with square brackets and transmits enclosed characters back to PC using interrupts

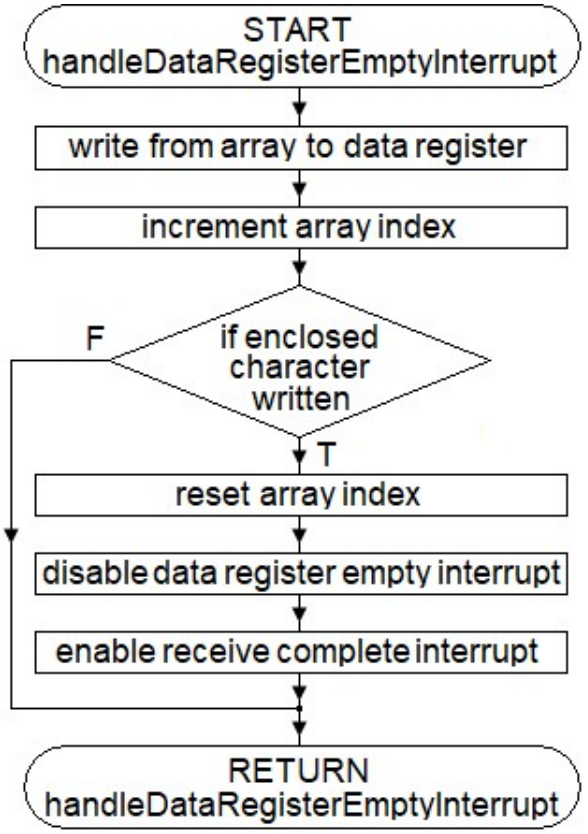
Firmware

- Algorithm









C Program

```
1  // charEncloserUsingInterruptsC.c
2  #include <avr/io.h>                          // I/O definitions
3  #include <avr/interrupt.h>                   // definitions of interrupt routines
4  char ch[] = {'[', ' ', ']'};                 // character array with enclosing brackets
5  volatile int i = 0;                          // array index;
6  int main(void)
7  {
8      UBRR0 = 103;                             // set baud rate as 9600bps
9      UCSR0B = UCSR0B & 0b11111011;            //
10     UCSR0C = UCSR0C | 0b00000110;            // specify char size as 8-bit
11     UCSR0B = UCSR0B | 0b00011000;            // enable RX & TX
12     UCSR0B = UCSR0B | 0b10000000;            // enable receive complete interrupt
13     sei();                                   // enable global interrupts
14     while (1)
15     {
16     }
17     return 0;
18 }
19 ISR(USART_RX_vect)                          // if receive is complete
20 {
21     ch[1] = UDR0;                            // read from data register to array
22     UCSR0B = UCSR0B & 0b01111111;            // disable receive complete interrupt
23     UCSR0B = UCSR0B | 0b00100000;            // enable data register empty interrupt
24 }
25 ISR(USART_UDRE_vect)                         // if data register is empty
26 {
27     UDR0 = ch[i++];                          // write from array to data register and increment index
28     if (i == 3)                              // if enclosed character written
29     {
30         i = 0;                               // reset array index
31         UCSR0B = UCSR0B & 0b11011111;        // disable data register empty interrupt
32         UCSR0B = UCSR0B | 0b10000000;        // enable receive complete interrupt
33     }
34 }
```

Summary

RECEIVE

bits received via RXD pin

↓

Receive Shift Register assembles bits

↓

character transferred to Receive Data Buffer

↓

UDR provides access to unread character

↓

RXC flag bit becomes 1

↓

if RXCIE is 1?

↓

USART_RX_vect ISR executes

↓

CPU reads character from UDR

↓

RXC flag bit becomes 0

TRANSMIT

Transmit Data Buffer is empty

↓

UDRE flag bit is 1

↓

if UDRIE is 1?

↓

USART_UDRE_vect ISR executes

↓

CPU loads new character to UDR

↓

UDRE flag bit becomes 0

↓

Transmit Data Buffer receives character

↓

character transferred to Transmit Shift Register

↓

UDRE flag bit becomes 1

↓

bits transmitted via TXD pin

Questions